The Long Boil · How it runs

A steampunk idle MMO for web and phone · live at thelongboil.com on AWS (us-west-2), deployed on every push

AWS · Our code · Outside service · Planned

AWS · us-west-2 · managed with CDK (TypeScript)

PLAYER
The game client
Expo + React Native Web, TypeScript. An installable web app (PWA) that runs on desktop and phone.

Pixel-art sprites, the queue, Story, the Exchange, chat.



play

ROUTE 53 · ACM
Domain and HTTPS
thelongboil.com DNS and its TLS certificate; www redirects by a CloudFront Function.

CLOUDFRONT
One front door
Serves the app from S3, and sends /api/* and the live WebSocket on to the game servers.

S3
The app and its art
The built web app, sprites and background paintings.

app + art

/api, ws

APPLICATION LOAD BALANCER
Holds connections
HTTP and WebSockets. Answers only CloudFront, which signs every request with a secret header.

ECR
Container images
One image per commit.

ECS FARGATE · GRAVITON (ARM)
The game server
Node.js 22, TypeScript, Fastify. The authoritative rules engine: idle work resolved offline, combat, drops, the Exchange order book, the story engine, guilds, chat, the Cog Shop.

Every change is an event in an ordered log, with seeded randomness, so any result can be replayed.

SQL

SECRETS MANAGER
Keys, never in code
Database, Google, Discord, Stripe.

AURORA POSTGRESQL · SERVERLESS V2
The source of truth
Tinkers, inventory, the event log, Exchange orders and fills, story qualities, guilds, chat, Cog Crate deliveries.

Scales to zero when dev is idle.

sign in

COGNITO
Sign-in
Hosted login with PKCE. The server checks a Cognito token on every request.

BUDGETS · EVENTBRIDGE · LAMBDA · SNS
Cost guard
A $100 budget alarm and a daily spend email.

OAuth

GOOGLE · DISCORD
Identity providers
Players sign in with an account they already have.

signed webhook

STRIPE (TEST MODE)
Cog Crate payments
Hosted Checkout takes the card. A signed webhook tells the server, which delivers each paid crate exactly once.

image

GITHUB ACTIONS
CI and deploy
Every push: 177 tests, 13 content validators, then a Docker build and CDK deploy. Signs in to AWS with OIDC, so no AWS keys are stored.

FAIR PLAY PROMISE
Cogs buy time, not power
Never story, Nerve or stats. Crates trade for in-game Crowns, so anyone can earn them.

checkout (card details go only to Stripe)

PLANNED ON AWS, IN THE DESIGN

Valkey (sessions, leaderboards) · AppConfig (content without a deploy) · EventBridge + SQS (scheduled jobs) · SES (email) · Admin console

Region and services from `infra/` (AWS CDK) · runtime from `engine/`

The Long Boil · How it is built

One developer, AI agents with one job each, deterministic checks between them, and a human sign-off on everything players see

Me · Claude agent · Code check · Gemini

Content: game data written by agents

items, monsters, drops, recipes, storylets, letters · all JSON in git, never hard-coded

ME
Brief and canon
Design doc, a work order per step, and a canon bible. Each agent sees only the story layers it is cleared for.

›

CLAUDE CODE · OPUS 5.5
Orchestrator
Builds the engine with me and hands bounded work to six specialist sub-agents.

›

DRAFTSMAN · SCRIVENER
Draft
Draftsman: numbers (stats, drops, recipes). Scrivener: story scenes and letters in the house voice.

›

13 VALIDATORS · PYTHON
Deterministic checks
Schema, stat budgets, drop odds, XP pacing, story graph reachability, reveal timing, banned words. Built by the Toolwright agent.

›

INSPECTOR
Review
What code cannot check: voice, lore, names, and whether a secret slips out too early.

›

ASSAYER
Simulate
Plays the numbers forward: pacing, Crown faucets and sinks, story pace.

›

ME
Approve
Nothing ships without my sign-off. Then git → CI → deploy.

↺ A draft that fails a check or the Inspector goes back to its author. · The **Registrar** agent records every item's history in a provenance register: 139 items, each with who drafted, checked, reviewed and approved it.

Art: pixel art drawn by Gemini

I write exact sheet specs; scripts cut them into game sprites

ME
Sheet spec
Prompt with a fixed grid, size and flat magenta background (`docs/art-requests.md`).

›

GOOGLE GEMINI
Draws the sheet



›

IMPORT_ART.PY
Slice and key out
Finds each cell, removes the magenta, checks frames line up (sheets often break the grid).

›

BUILD_SPRITES.PY
Into the game



›

PLAYWRIGHT
Screenshot check
Desktop and phone screenshots of every UI change before it ships.

Tech stack

Client
Expo 57, React Native Web, React 19, TypeScript

Server
Node.js 22, Fastify 5, WebSockets, a pure TypeScript rules engine

Data
PostgreSQL (Aurora), an event log, Docker Postgres locally

Infra
AWS CDK, Docker, GitHub Actions with OIDC

Quality
Vitest (177 tests against real Postgres), Python validators, Playwright

Next

in the design, not built yet

Agents move into AWS: **Step Functions** runs each agent as a Lambda calling Claude on **Amazon Bedrock**, with my approval as a task token. Two more agents: the **Foreman** (turns a patch brief into work orders) and the **Chronicler** (writes the in-game newspaper from game data).

Agents and validators from `.claude/agents/` and `scripts/validate/` · art from `scripts/art/`